Building and Adapting Small Legal Language Models for Indian Legal NLP Tasks

CS781 Capstone Project Report
Madhav Dwivedi (Roll No. _____)
GitHub: _____

Abstract

In this project we pretrained four small language models (70M to 94M parameters) from scratch on the legal corpus given in the competition and then used them for six legal NLP tasks: named entity recognition, rhetorical role labelling, judgment prediction, statute identification, summarisation and prior case retrieval. In each phase we tried all four models with different fine-tuning methods, including LoRMA and knowledge distillation which were compulsory. Our decoder-only model (A_v2, 94M) turned out to be the best backbone in all phases. On this model full fine-tuning, LoRA and LoRMA gave very similar scores, but LoRMA did not train properly on the encoder-decoder and the hybrid model in several phases. Distillation into a 6-layer student helped by about 1.5 to 3 F1 points in the classification phases. Our best Codabench scores were 0.8334 (P1), 0.5194 (P2), 0.5150 (P3), 0.7350 (P4), 0.3870 (P5), 0.8421 (P6) and 0.3768 (P7); in phases 4 to 7 the best one was an ensemble of a few models.

1 Introduction

Indian court judgments are very long documents, often more than 10,000 tokens, and they use a lot of legal terms, section numbers and case citations. The aim of this capstone was to check whether a small model that we pretrain ourselves only on legal text can be useful for the tasks from the IL-TUR benchmark [1]. There were some rules that affected almost every decision we made:

- From Phase 2 onwards we could only start from our own Phase 1 models, no pretrained weights from outside and no LLM APIs.
- All four Phase 1 architectures had to be used in every phase, and the best four submissions had to be from different models.
- Every phase needed at least 4 different experiments, with at least 3 fine-tuning techniques (LoRMA was compulsory) and at least one knowledge distillation experiment.

We only had Kaggle's free GPUs (2 x T4) and every phase was open for about five days, so time was the main constraint. Because of this, after Phase 2 we wrote one common pipeline and kept reusing it. It loads a Phase 1 checkpoint, applies a fine-tuning method from our own `peft.py` file and adds a task-specific head. For every phase there is one Kaggle notebook that trains around 8 to 15 configurations (two at a time, one per GPU), saves the dev scores in a `RESULTS.csv` file and makes the submission zip files. The rest of the report goes through Phase 1, the fine-tuning methods we used, each downstream phase, and finally what we learned overall.

2 Phase 1: Pretraining the Legal Language Models

In Phase 1 we had to train a language model with 50M to 300M parameters from scratch. The test set has 512-word prompts and the model has to write the next 128 words. The main metric is BERTScore F1, with ROUGE-L and BLEU-4 as secondary metrics, and at least four clearly different architectures were required.

Data and tokenizer. We used the byte-level BPE tokenizer from the course assignment (32,000 merges). The corpus has more than 4.5B tokens, which was too much for our compute, so we sampled a mix of 35% Supreme Court, 25% High Courts, 20% Acts and 20% Tribunals. We tokenised about 500M tokens and trained on 400M of them. A small part was kept aside as validation data, and we also made a local test of 300 prompts so that we could compute ROUGE-L and BLEU without submitting. All models were trained with AdamW (learning rate 6e-4 with cosine decay to 6e-5, 500 warmup steps, weight decay 0.1) and an effective batch of 64 sequences. Since Kaggle sessions are limited, each model was trained for a fixed time of about 10 hours instead of a fixed number of steps.

Architectures. We tried the following four designs:

- **A: decoder-only** in the LLaMA style, with RMSNorm, rotary position embeddings (RoPE), SwiGLU feed-forward layers and grouped-query attention. Later we also trained a bigger version, **A_v2**, with 11 layers of

width 768 on two GPUs.

- **B: encoder-decoder** similar to T5. The first part of a text goes to the encoder and the decoder predicts the rest.
- **C: hybrid (UniLM style)**. One shared transformer is trained as a prefix-LM (the prefix is read bidirectionally and the rest causally), and 20% of the samples use BERT-style masking. We chose this because we expected it to be a good encoder later.
- **D: efficient decoder** with sliding-window attention (each token sees only the last 256 tokens) and multi-query attention (one key/value head). This makes attention cheaper, so D could be trained with 2048-token sequences.

Model	Layers x width	Params	Context	Val loss	ROUGE-L	BLEU-4	BERTScore (test)
A (decoder)	12 x 640	72M	1024	1.936	0.287	17.7	0.8310
B (enc-dec)	6+6 x 576	74M	768 / 384	2.207	0.177	3.4	0.8210
C (hybrid)	10 x 640	70M	1024	2.132	0.274	16.2	0.8293
D (SWA + MQA)	12 x 640	71M	2048	1.984	0.274	16.3	0.8304
A_v2 (decoder)	**11 x 768**	**94M**	**1024**	**1.955**	**0.289**	**17.8**	**0.8334**

Table 1: Phase 1 models. ROUGE-L and BLEU-4 are from our local 300-prompt test, BERTScore is from Codabench.

For generation we used sampling with temperature 0.8, top-k 50, top-p 0.95 and a repetition penalty of 1.1. For A_v2 we also tried a few other settings (lower temperature, greedy with no-repeat n-grams). Greedy gave a slightly higher BLEU but the text became repetitive and less natural, so we kept sampling.

A_v2 got the best score (0.8334). One thing we noticed is that BERTScore hardly separates the models (all are between 0.821 and 0.834) while ROUGE-L and BLEU show much bigger differences. B was clearly the weakest. When we looked into it, the reason was mostly our training setup: only about one third of every sample was used as the decoder target, so B was trained on about 133M target tokens compared to 400M for A. Its decoder also starts from an empty `<bos>` token, so the continuation often does not connect well with the end of the prompt. From Phase 2 onwards we used A_v2 as our "A" model. For B we mostly used only the encoder (42M) in the classification tasks.

3 Fine-tuning Methods

Since we could not use libraries like HuggingFace PEFT with our own architectures, we implemented all methods ourselves in `peft.py`. Every method is initialised so that at the start of training the model gives exactly the same output as the pretrained model, and we checked this with small unit tests. Below, W_0 is a frozen pretrained weight matrix and x is the input of that layer. To compare sizes we give the number of trainable weights for one 640 x 640 layer (which has 409,600 weights).

- **Full fine-tuning:** all weights are trained, with a small learning rate for the backbone (2e-5 to 5e-5) and a larger one for the new head. 409,600 trainable weights per layer.
- **Partial fine-tuning:** only the last k transformer blocks are trained and the rest are frozen.
- **LoRA [3]:** a low-rank matrix is added to the frozen weight: $h = W_0x + (\alpha/r)BAx$, where A is r x d and B is d x r and B starts at zero. With r = 8 this is 10,240 weights per layer (2.5%). We also used rsLoRA [11], which scales by $\alpha/\sqrt{r}$ instead of α/r.
- **LoRMA [12]:** instead of adding, the output of the frozen layer is multiplied by a low-rank matrix: $h = (I + (\alpha/r)BA)W_0x$. We compute it as $B(A(W_0x))$ so the big d x d matrix is never built. It has the same number of parameters as LoRA.
- **QLoRA [9]:** the frozen weights are stored in 4-bit NF4 format (we wrote our own quantiser) and LoRA with r = 16 is trained on top.
- **AdaLoRA [10]:** the update is written as P diag(λ) Q like an SVD, and the less important singular values are pruned during training.
- **Adapters [6, 7]:** a small bottleneck MLP (down to 64 dimensions and back up) is added after the attention and FFN (Houlsby) or only after the FFN (Pfeiffer).
- **IA³ [13]:** learned vectors that scale the keys, values and FFN activations. This has the fewest parameters of all.

- **Prefix tuning [4]:** some learned key/value vectors are added in every layer so that the real tokens can attend to them, while the backbone is frozen.
- **Knowledge distillation [2]:** a smaller student (the first 4 or 6 layers of a Phase 1 model) is trained on both the gold labels and the soft outputs of a fine-tuned teacher, using $L = \alpha\, CE + (1-\alpha)\, T^2\, KL(p_{teacher} \,||\, p_{student})$ with $T = 2$.

Difference between LoRA and LoRMA. It took us some time to understand how these two really differ, so here is the small example we worked out. Suppose $W_0x = [2, -1]$, the input is $x = [1, 1]$, and the rank-1 update has $A = [0.5, 0]$ and $B = [1, 0]^T$. LoRA adds $BAx = [0.5, 0]$, so $h = [2.5, -1]$. LoRMA multiplies the output instead: $(I + BA)[2, -1] = [2 + 0.5 \cdot 2, -1] = [3, -1]$. So in LoRMA the change depends on what W_0 already outputs. It can scale and mix the existing features but it cannot easily create new ones. We think this explains a lot of our LoRMA results (Section 5): it works well when the pretrained features are already good, and badly when they are not.

Phase	A_v2	B	C	D	KD (teacher to student)
2 NER	LoRA	LoRMA	full, full + CRF	Houlsby adapters	C full to C 4-layer
3 RR	LoRA (+CRF), LoRMA, prefix, full	rsLoRA	LoRA + CRF, LoRMA	IA³	A_v2 to A 4-layer, C to C 4-layer
4 CJPE	full, LoRMA	adapters, full	LoRMA	partial	A_v2 full to D 6-layer
5 LSI	full (+LWAN, +ASL), LoRMA, QLoRA	full, adapters	partial, LoRMA	student	A_v2 full to D 6-layer
6 Summ.	full, LoRMA (+DAPT), QLoRA, prefix	LoRA, Houlsby	Pfeiffer, AdaLoRA	student	A_v2 full to D 6-layer
7 PCR	full (+MaxSim, +ANCE), LoRMA, QLoRA, prefix	full, IA³	partial	student	A_v2 full to D 6-layer

Table 2: Which methods we ran on which model in each phase.

4 Experiments and Results

For every phase we kept a fixed dev split (the official one if it was given, otherwise a part of the training data) and selected models and thresholds only on it. In the tables "Dev" is our own dev score and "Test" is the Codabench score. A dash means we did not submit that run. The best submission of each phase is in bold.

4.1 Phase 2: Legal Named Entity Recognition

The task is to find entities such as appellant, respondent, judge, court, statute, precedent and date in full judgments and return their exact character positions. The metric is strict macro-F1, so a span that is off by even one character counts as wrong.

We treated it as token classification with BIO tags (25 tags) and a linear head, and tried a CRF on top for one run. The hard part was mapping BPE tokens back to characters. We used the byte offsets from our tokenizer and checked that converting the gold spans to tags and back gives 0.994 F1, so the tokenization itself does not lose much. Documents were split into 512-token windows with some overlap. One thing we changed early was to run A and D with a bidirectional attention mask, because with a causal mask a token cannot see the next words and the model has trouble deciding where an entity ends. (Also, at the start of this phase our B checkpoint was missing and we had to download it again from the Phase 1 run.)

Run	Model	Method	Trainable	Dev	Test
E3	**A_v2 (bidirectional)**	**LoRA, r = 8**	**1.1%**	**0.579**	**0.5194**
E6	C	full + CRF	100%	0.557	0.5171
E1	C	full, linear head	100%	0.569	0.5168
E4	D	Houlsby adapters	2.7%	0.474	0.4093
E5	C, 4 layers	KD from E1	100%	0.459	0.3879
E2	B encoder	LoRMA	2.5%	0.225	0.2623

Table 3: Phase 2 results.

The best submission was A_v2 with LoRA (0.5194), even though only about 1% of its weights were trained. C with full fine-tuning was close behind, probably because the masked-LM part of its pretraining already taught it to

read in both directions. The CRF did not help much because our BIO-constrained decoding was already removing most invalid tag sequences. LoRMA on B's encoder did very badly (0.26). Since LoRMA only rescales what the frozen layer outputs, a weak encoder like B does not give it much to work with.

4.2 Phase 3: Rhetorical Role Prediction

Here every sentence of a judgment has to be labelled with its role (facts, issues, arguments, precedent, ratio, ruling and so on), for two domains, corporate law and IT law. The metric is macro-F1.

Our first version (the R runs) labelled each sentence using a 512-token context. We then made it hierarchical: the document is encoded in windows of up to 1,024 tokens, every sentence is mean-pooled into one vector, a bidirectional GRU runs over the sequence of sentence vectors and a CRF predicts the role sequence. The idea is that the role depends a lot on the position and on the neighbouring sentences (facts come at the beginning, the ruling at the end). We also tried "refit", where after choosing the best epoch on dev we train again on train + dev. Our very first Kaggle run of this phase crashed at the settings cell because of a small JSON error, so we started testing the notebooks locally before uploading them.

Run	Model	Method	Dev	Test
V1 refit	**A_v2**	**LoRA + sentence GRU + CRF, 1k context**	**0.569**	**0.5150**
V4 refit	C	LoRA + GRU + CRF	0.566	0.5141
V1	A_v2	same as above, no refit	0.569	0.4971
V3	A_v2	prefix tuning	0.484	0.4616
V5	A_v2, 4 layers	KD from V1	0.480	0.4605
R3	B	rsLoRA	0.487	–
R5 / no KD	C, 4 layers	KD from R1 / same without KD	0.491 / 0.465	–
R4	D	IA^3	0.431	–
V2	A_v2	LoRMA	0.368	0.2964

Table 4: Phase 3 results. The R runs are from our first version and were only evaluated on dev.

The best submission was A_v2 with LoRA, GRU and CRF after refit (0.5150). The sentence context made the biggest difference: without the GRU the same model got only 0.357 on dev, compared to 0.523 with it. The CRF and the longer context added about 4.5 more points, and refitting on train + dev gave 1.8 more on the test set. LoRMA was again a problem. It stopped improving after the first epoch, and the refit version of it collapsed to predicting almost only one class (0.04 on test).

4.3 Phase 4: Court Judgment Prediction

The task is to predict from the case text whether the appeal was accepted or rejected (binary), scored with macro-F1. While preparing the data we found 5,082 repeated case ids in the training set and removed them, leaving 32,305 cases.

The cases are much longer than our context length, so we had to decide which parts to read. We took four chunks of 512 tokens from the beginning and the end of the document, because the end of a judgment usually has the reasoning and the decision. Each chunk is pooled into a vector, the chunks are combined with attention pooling, and a linear layer gives the output. The decision threshold was tuned on dev. D could read 4 chunks of 1,024 tokens because its attention is cheaper.

Run	Model	Method	Trainable	Dev	Test
ENS	**R1 + X1 + R4**	**average of probabilities**	**–**	**0.725**	**0.7350**
R4	D, 6 layers	KD from R1	100%	0.706	0.7126
X1	A_v2	LoRMA	1.4%	0.707	0.7098
R1	A_v2	full	100%	0.711	–
X4	D, 6 layers	same student, no KD	100%	0.679	0.6800
X3	A_v2	full, only last 512 tokens	100%	0.677	0.6782
X2	D	partial (top layers)	24%	0.638	0.6426
X5	B	full	100%	0.643	0.6412
R2	B	adapters	2.2%	0.577	–
R3	C	LoRMA	1.6%	0.504	0.4952

Table 5: Phase 4 results.

Our best submission was the average of three models: A_v2 full fine-tuning, A_v2 with LoRMA and the distilled D student (0.7350). This was the phase where LoRMA worked best for us: on A_v2 it almost matched full fine-tuning while training only 1.4% of the weights. Reading both the beginning and the end of the case was 3.4 points better than reading only the last 512 tokens. Distillation also clearly helped: the 6-layer D student went from 0.680 without KD to 0.713 with KD on the test set. The three models in the ensemble make somewhat different mistakes, and averaging them added 2.2 points over the best single submission.

4.4 Phase 5: Legal Statute Identification

For each case we have to predict which statutes it refers to. There are 100 labels and a case has about 3.9 of them on average. The metric is macro-F1 over the statutes.

We reused the document encoder from Phase 4 and changed the output to one sigmoid per statute with BCE loss (we also tried the asymmetric loss [16]). The thresholds turned out to be very important. With a fixed threshold of 0.5 the model predicted only 1.8 statutes per case and macro-F1 was 0.25, because rare statutes almost never crossed 0.5. So we tuned one threshold per statute. To avoid overfitting the dev set we did this with 2-fold cross-fitting (tune on one half of dev, evaluate on the other half), and macro-F1 went up to about 0.39.

Run	Model	Method	Dev	Test
ENS	**T1 + X2 + R4**	**average + per-statute thresholds**	**0.405**	**0.3870**
T1	A_v2	full (teacher for KD)	0.400	0.3805
X2	A_v2	full + label-wise attention head	0.399	0.3793
R4	D, 6 layers	KD from T1	0.394	0.3763
X4	A_v2	full + asymmetric loss	0.392	0.3750
X3	A_v2	QLoRA	0.390	0.3713
R3	C	partial (last 4 blocks)	0.381	0.3644
R1	A_v2	LoRMA	0.381	0.3642
X1	D, 6 layers	no KD	0.383	0.3611
R2	B	full	0.332	0.3131
X5 / X6	B / C	adapters / LoRMA	0.253 / 0.081	–

Table 6: Phase 5 results.

The ensemble was again the best (0.3870), and the best single model was A_v2 with full fine-tuning (0.3805). Most runs ended up between 0.36 and 0.38, so we think the limit here is the rare statutes that have very few training examples rather than the choice of model. KD gave the D student 1.5 points on test. LoRMA on A_v2 was 1.6 points below full fine-tuning, and LoRMA on C failed completely: it ended up predicting around 25 statutes for every case.

4.5 Phase 6: Summarisation

In this phase the model has to write a summary of a judgment. The reference summaries are long (about 900 words on average). The metric is BERTScore F1, and we also looked at ROUGE.

For A, C and D we gave the model the input "document SUMMARY: summary" and computed the loss only on the summary part. For C the document part is read bidirectionally, the same way as in its prefix-LM pretraining. For B we used a Fusion-in-Decoder setup [15]: the encoder reads 4 chunks of 768 tokens separately and the decoder attends to all of them. The documents are too long to fit, so we cut them to 1,400 tokens, either by taking the head and the tail or by a "salient" method that keeps the first 256 tokens and then adds the sentences with the highest importance score. We used greedy decoding with a rule that blocks repeated 4-grams. We also tried continuing the language-model training on the training documents for 300 steps before fine-tuning (DAPT), and combining several models. For combining we used MBR: for each document we pick the summary that is most similar (ROUGE-1/2) to the summaries from the other models. We also trained a reward model to pick the best summary out of 15 candidates.

Run	Model	Method	Dev BERTScore	Dev ROUGE-L	Test
ENS_MBR	**X5, R1, X2, T1**	**MBR selection**	**0.842**	**0.244**	**0.8421**
X5	A_v2	LoRMA + salient sentences	0.843	0.245	0.8407
R1	A_v2	LoRMA, head + tail	0.841	0.241	0.8399
X7	A_v2	DAPT + LoRMA	0.840	0.242	0.8381
X6	C	AdaLoRA	0.837	0.233	0.8367
X3	A_v2	prefix tuning	0.832	0.227	0.8290
ENS_RM	15 candidates	reward model reranking	–	0.215	0.8267
X8	B	LoRA, 768 tokens	0.817	0.197	0.8173
X4	B	Houlsby, FiD 3k tokens	0.817	0.198	0.8171
T1 / X2	A_v2	full / QLoRA	0.841 / 0.841	0.240 / 0.238	–
R3	C	Pfeiffer adapter	0.838	0.236	–
R4 / X1	D, 6 layers	KD from T1 / no KD	0.821 / 0.823	0.204 / 0.204	–

Table 7: Phase 6 results (dev = 200 documents held out from the training set).

The MBR ensemble gave our best score (0.8421), and the best single model was A_v2 with LoRMA and salient sentence selection (0.8407). For summarisation LoRMA on A_v2 was as good as full fine-tuning (both 0.841 on dev). We think this is because writing a summary is still next-token prediction, which is what the model was pretrained for, so it mostly needs to re-weight what it already knows. Choosing salient sentences worked a little better than a fixed head and tail cut. DAPT did not help.

The reward model was disappointing. We trained it (C with LoRMA, Bradley-Terry loss) on 3,240 pairs of summaries, where the better summary was decided by ROUGE against the reference. It reached only 58% pairwise accuracy and its scores had basically no correlation with ROUGE (Kendall $\tau = 0.003$), so its choices were close to random and the submission was worse than our single models. Distillation also did not help here. Our summaries are about half as long as the references (about 430 vs. 900 words), and the main problem of the student was that it missed facts from the document (fact recall 0.05 vs. 0.23 for A_v2). Matching the teacher's token probabilities does not fix that.

4.6 Phase 7: Prior Case Retrieval

For each query case we have to find the earlier cases it cites from a pool of candidates and return the top 7. The metric is micro-F1@7. There are 827 training queries and 118 dev queries. On dev a query cites 8.2 cases on average, so even a perfect system could only reach about 0.62 F1@7, because only 7 cases can be returned.

We used a bi-encoder: the query and each candidate are encoded into one vector each (with the chunk pooling from Phase 4 and a projection layer), and the similarity is the cosine between them. We trained it with a contrastive (InfoNCE) loss, where the cited cases are the positives and the negatives are other cases in the batch plus "hard" negatives found with BM25. We also tried some variants: MaxSim, where every query chunk is matched to its most similar candidate chunk; ANCE, where the hard negatives are found again by the model itself; and a pairwise ranking loss. For KD the D student learns to copy the teacher's similarity scores. Finally, since BM25 alone was already quite strong, we combined the dense scores with BM25 (after normalising the scores per query). HYB is one dense model + BM25, and ENS is three dense models + BM25. We also tried removing candidates that are dated after the query, but this made dev worse, because by our year detection about 21% of the gold cases seem to have a later date.

Run	Model	Method	Trainable	Dev F1@7	Test
ENS	**X7 + R3 + X8**	**dense scores + BM25 (weight 0.3)**	**–**	**0.388**	**0.3768**
HYB	X7	dense + BM25 (weight 0.4)	–	0.381	0.3698
X7	A_v2	full, attention pooling + MaxSim	100%	0.326	–
R3	C	partial (last 4 blocks)	28%	0.320	–
X8	A_v2	full + ANCE negatives	100%	0.315	–
X2	A_v2	QLoRA	2.4%	0.311	–
BM25	–	word + bigram matching only	0	0.310	–
T1	A_v2	full (teacher for KD)	100%	0.308	–
R4 / X1	D, 6 layers	KD from T1 / no KD	100%	0.304 / 0.282	–
X3	A_v2	prefix tuning	0.35%	0.304	–
X5 / X6	A_v2	only in-batch negatives / pairwise loss	100%	0.289 / 0.256	–
R1	A_v2	LoRMA	2.7%	0.285	0.2634
X0	A_v2	no training (frozen Phase 1 model)	0	0.261	–
R2 / X4	B	full / IA³	100% / 0.4%	0.259 / 0.241	0.2126 / –

Table 8: Phase 7 results.

The best submission was ENS (0.3768). The most surprising result for us was how strong BM25 is for this task: with bigrams it gets 0.310, about the same as most of our trained dense models. But BM25 and the dense models seem to get different queries right, so combining them gave the biggest improvement in this phase, about 5.5 points on dev compared to the best dense model alone. The frozen A_v2 model without any training (0.261) was already about as good as B after full fine-tuning, which again shows how much depends on the pretrained model. MaxSim was the best dense variant. We think the reason is that a judgment usually cites a case in only a few paragraphs, and averaging over the whole document hides that. BM25 hard negatives were better than in-batch negatives only (+1.9), and KD gave the D student 2.1 points. LoRMA was 2.3 points behind full fine-tuning here.

5 Discussion

Phase	Metric	Best submission	Test
1 Pretraining	BERTScore F1	A_v2 (94M decoder)	0.8334
2 NER	strict macro-F1	A_v2 + LoRA	0.5194
3 RR	macro-F1	A_v2 + LoRA + GRU + CRF, refit	0.5150
4 CJPE	macro-F1	ensemble (A_v2 full, A_v2 LoRMA, D with KD)	0.7350
5 LSI	macro-F1	ensemble (A_v2 full, A_v2 LWAN, D with KD)	0.3870
6 Summarisation	BERTScore F1	MBR over 4 A_v2 runs	0.8421
7 PCR	micro-F1@7	3 dense retrievers + BM25	0.3768

Table 9: Best submission in each phase.

The pretrained model mattered more than the fine-tuning method. A_v2, which had the best Phase 1 numbers, was the best backbone in every single phase, and B, which was weakest in Phase 1, was at or near the bottom every time. On A_v2 the difference between full fine-tuning, LoRA, LoRMA and QLoRA was usually 2 points or less, while changing the backbone with the same method often changed the score by 5 to 20 points. If we had more time, we would spend it on better pretraining rather than on more fine-tuning tricks.

Which fine-tuning method worked best. There was no single winner. Full fine-tuning was the safest choice for the document classification tasks (Phases 4 and 5), where the task is quite different from language modelling. LoRA was best for NER and rhetorical roles, where the training data is small (a few hundred documents) and training only 1 to 3% of the weights avoided overfitting. LoRMA was close to full fine-tuning on A_v2 in judgment prediction and was our best single method for summarisation, but it was behind in retrieval and failed on B and C in several phases. Our explanation, based on the example in Section 3, is that LoRMA can only rescale and mix what the frozen layer already produces. With a strong model and a task close to pretraining this is enough, but with a weak model it is not, and since the size of the update grows with the size of W_0x, training also became unstable (in three of the four failed runs the best epoch was the first one). We would like to try a lower learning rate or the LoRMA_π variant for these models. Prefix tuning and adapters were usually 2 to 6 points behind, and IA³ was the weakest.

Knowledge distillation helped for classification and retrieval. With the same 6-layer D student, KD added 3.3 points in Phase 4, 1.5 points in Phase 5 and 2.1 points on dev in Phase 7, and in Phase 3 the 4-layer C student gained 2.5 points on dev. The student is half the size and was 1.2 to 1.4 times faster in our runs. In summarisation KD did not help.

The task-specific design was as important as the method. Many of our biggest improvements came from how we read the input and made the decisions: bidirectional attention for NER, the sentence-level GRU for rhetorical roles (+17 dev points), reading the head and tail of the case in Phase 4 (+3.4), per-statute thresholds in Phase 5 (+14 over a 0.5 threshold), and adding BM25 in Phase 7 (+5.5 on dev). These gains were often bigger than the differences between fine-tuning methods.

What did not work. LoRMA on B and C, the refit of an unstable LoRMA run, the reward model in Phase 6 (300 documents and ROUGE-based labels were not enough to train it), DAPT, and in general the B encoder-decoder, which never recovered from its weaker pretraining.

6 Limitations and Future Work

- We pretrained on only 400M tokens, which is less than 10% of the corpus, because of the 10-hour limit per model. Training A_v2 for longer would probably help all the later phases.
- We only read 2,000 to 4,000 tokens of documents that are often more than 10,000 tokens long. A model that reads the whole document hierarchically, or pretraining D with a longer context, could help judgment prediction, statute identification and retrieval.
- In some phases the dev set came from the training data, and some improvements on dev did not fully carry over to the test set. We also ran every configuration with only one seed because of the GPU limits, so small differences (below 1 point) may be noise.
- Our summaries are about half as long as the reference summaries. Length control during decoding, or a better reward model trained on more data, would be the next things to try.
- LoRMA needs a more stable training setup to work on all four architectures.

7 Conclusion

We showed that small legal language models that we trained ourselves on free Kaggle GPUs can be adapted to six quite different legal tasks without any outside pretrained weights. Our 94M decoder A_v2 was the best base model in all phases. On top of it LoRA worked best for token and sentence labelling, full fine-tuning for document classification, and LoRMA for summarisation, where it matched full fine-tuning with only 2.5% of the weights trained. Distillation gave us students half the size that kept most of the accuracy, and combining a few different models (and BM25 for retrieval) gave our final improvement in Phases 4 to 7.

LLM Declaration

[To be filled using the questionnaire in the course report template.]

References

[1] A. Joshi et al. IL-TUR: Benchmark for Indian Legal Text Understanding and Reasoning. arXiv:2407.05399, 2024.
[2] G. Hinton, O. Vinyals, J. Dean. Distilling the Knowledge in a Neural Network. 2015.
[3] E. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. 2021.
[4] X. L. Li, P. Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. 2021.
[5] B. Lester, R. Al-Rfou, N. Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. 2021.
[6] N. Houlsby et al. Parameter-Efficient Transfer Learning for NLP. 2019.
[7] J. Pfeiffer et al. AdapterFusion: Non-Destructive Task Composition for Transfer Learning. 2021.
[8] E. Ben Zaken, Y. Goldberg, S. Ravfogel. BitFit. 2022.
[9] T. Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. 2023.
[10] Q. Zhang et al. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. 2023.
[11] D. Kalajdzievski. A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA. 2023.
[12] B. Bihany, A. Patel, A. Modi. LoRMA: Low-Rank Multiplicative Adaptation for LLMs. Findings of ACL, 2025.
[13] H. Liu et al. Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning. 2022.
[14] J. Lafferty, A. McCallum, F. Pereira. Conditional Random Fields. 2001.
[15] G. Izacard, E. Grave. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. 2021.
[16] T. Ridnik et al. Asymmetric Loss for Multi-Label Classification. 2021.